

Programación para Física y Astronomía

Departamento de Física.

Coordinadora: C Loyola

Profesores C Femenías / D Basantes

Primer Semestre 2025

Universidad Andrés Bello

Departamento de Física y Astronomía



Introducción y Repaso

Fundamentos de Matplotlib

Matplotlib Avanzado

Gráficos 3D Básicos

Introducción a pandas (Opcional)

Ejercicios Prácticos

Soluciones de Referencia

Conclusiones y Próximos Pasos

Introducción y Repaso

- **Semana 6, Sesión 2 (Sesión 12)** se enfocó en:
 - Manipulación avanzada de NumPy (**reshape**, **transpose**, **concatenate**).
 - Uso de **np.random** para valores aleatorios y **np.linalg** (álgebra lineal).
 - Primeros pasos con **Matplotlib** (**plot**, **scatter**).
- **Objetivo de hoy:** Avanzar con gráficos más complejos (subplots, histogramas, 3D) y comenzar a manipular datos de forma más eficiente (posiblemente una introducción a **pandas**).

- **Consolidar** los fundamentos de **Matplotlib** con guías paso a paso.
- **Profundizar** en las opciones de **Matplotlib** para visualizar datos:
 - Subplots, estilos, anotaciones.
 - Gráficos de barras, histogramas y 3D simples.
- **Familiarizarnos** con un primer acercamiento a **pandas**, si el tiempo lo permite.
- **Ejercitar** estos conceptos con ejemplos y datos relevantes para Física/Astronomía.

Fundamentos de Matplotlib

- **Librería de visualización** más popular en Python para crear gráficos estáticos, animados e interactivos.
- **Inspirada en MATLAB:** sintaxis similar y familiar.
- **Dos interfaces principales:**
 - `pyplot` (estilo MATLAB, más simple)
 - Orientada a objetos (más control y flexibilidad)
- **Integración perfecta** con NumPy y otras librerías científicas.

Importación estándar

```
import matplotlib.pyplot as plt
```

```
1 # Paso 1: Importar librerías necesarias
2 import matplotlib.pyplot as plt
3 import numpy as np
4
5 # Paso 2: Crear datos
6 x = [1, 2, 3, 4, 5]
7 y = [2, 4, 6, 8, 10]
8
9 # Paso 3: Crear la gráfica
10 plt.plot(x, y)
11
12 # Paso 4: Mostrar la gráfica
13 plt.show()
```

- `plt.plot()`: función básica para gráficos de línea
- `plt.show()`: muestra la gráfica en pantalla

Fundamentos de Matplotlib $\ni$ Mejorando la Gráfica - Etiquetas y Títulos

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 x = [1, 2, 3, 4, 5]
5 y = [2, 4, 6, 8, 10]
6
7 plt.plot(x, y)
8
9 # Agregar etiquetas y título
10 plt.xlabel("Tiempo (s)")           # Etiqueta eje X
11 plt.ylabel("Posición (m)")         # Etiqueta eje Y
12 plt.title("Movimiento Rectilíneo Uniforme") # Título
13 plt.show()
```

- Siempre incluir unidades en las etiquetas
- Títulos descriptivos ayudan a la interpretación

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 x = np.linspace(0, 10, 50)
5 y1 = np.sin(x)
6 y2 = np.cos(x)
7
8 # Diferentes estilos de línea
9 plt.plot(x, y1, 'r-', linewidth=2, label='sen(x)') # rojo, sólido
10 plt.plot(x, y2, 'b--', linewidth=2, label='cos(x)') # azul, punteado
11
12 plt.xlabel("x (radianes)")
13 plt.ylabel("f(x)")
14 plt.title("Funciones Trigonómicas")
15 plt.legend() # Mostrar leyenda
16 plt.grid(True) # Agregar grilla
17 plt.show()
```

- Códigos de estilo: 'r-' (rojo sólido), 'b-' (azul punteado), 'g:' (verde puntos)
- `plt.legend()`: muestra las etiquetas definidas con `label`

Fundamentos de Matplotlib $\ni$ Tipos de Gráficos Básicos

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3 # Datos de ejemplo
4 x = np.linspace(0, 5, 20)
5 y = x**2
6 # 1. Gráfico de línea
7 plt.figure(figsize=(12, 4))
8 plt.subplot(1, 3, 1)
9 plt.plot(x, y, 'b-o')
10 plt.title("Gráfico de Línea")
11 plt.xlabel("x")
12 plt.ylabel("x²")
13
14 # 2. Gráfico de puntos (scatter)
15 plt.subplot(1, 3, 2)
16 plt.scatter(x, y, color='red', alpha=0.6)
17 plt.title("Gráfico de Dispersión")
18 plt.xlabel("x")
19 plt.ylabel("x²")
20
21 # 3. Gráfico de barras
22 plt.subplot(1, 3, 3)
23 categorias = ['A', 'B', 'C', 'D']
24 valores = [3, 7, 2, 5]
25 plt.bar(categorias, valores, color='green')
26 plt.title("Gráfico de Barras")
27 plt.xlabel("Categoría")
28 plt.ylabel("Valor")
29
30 plt.tight_layout()
31 plt.show()
```

Fundamentos de Matplotlib $\ni$ Configurando Ejes y Escalas

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 x = np.linspace(0, 2*np.pi, 100)
5 y = np.sin(x)
6
7 plt.plot(x, y, 'b-', linewidth=2)
8
9 # Configurar límites de los ejes
10 plt.xlim(0, 2*np.pi)
11 plt.ylim(-1.2, 1.2)
12
13 # Configurar marcas en los ejes
14 plt.xticks([0, np.pi/2, np.pi, 3*np.pi/2, 2*np.pi],
15            ['0', ' $\pi/2$ ', ' $\pi$ ', ' $3\pi/2$ ', ' $2\pi$ '])
16 plt.yticks([-1, -0.5, 0, 0.5, 1])
17
18 plt.xlabel("Ángulo")
19 plt.ylabel("sen(x)")
20 plt.title("Función Seno con Ejes Personalizados")
21 plt.grid(True, alpha=0.3)
22 plt.show()
```

- `plt.xlim()`, `plt.ylim()`: controlan los rangos de visualización
- `plt.xticks()`, `plt.yticks()`: personalizan las marcas en los ejes

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 x = np.linspace(0, 10, 100)
5 y = np.exp(-x/5) * np.cos(x)
6
7 plt.figure(figsize=(8, 6))
8 plt.plot(x, y, 'r-', linewidth=2)
9 plt.xlabel("Tiempo (s)")
10 plt.ylabel("Amplitud")
11 plt.title("Oscilación Amortiguada")
12 plt.grid(True, alpha=0.3)
13
14 # Guardar la figura
15 plt.savefig('oscilacion_amortiguada.png', dpi=300, bbox_inches='tight')
16 plt.savefig('oscilacion_amortiguada.pdf', bbox_inches='tight')
17 plt.show()
```

- `plt.savefig()`: guarda la gráfica en varios formatos
- `dpi`: resolución (dots per inch)
- `bbox_inches='tight'`: ajusta márgenes automáticamente

Fundamentos de Matplotlib $\ni$ Ejercicio Guiado:

Gráfica de Caída Libre



Enunciado

Crear una gráfica que muestre la posición vs tiempo para un objeto en caída libre:

- Usar la fórmula: $y(t) = y_0 - \frac{1}{2}gt^2$
- $y_0 = 100$ m (altura inicial)
- $g = 9.81$ m/s² (gravedad)
- Tiempo desde 0 hasta cuando el objeto toca el suelo

Pasos a seguir:

1. Crear array de tiempo
2. Calcular posiciones
3. Crear gráfica con etiquetas apropiadas
4. Agregar título y grilla
5. Marcar el punto donde toca el suelo

Fundamentos de Matplotlib $\ni$ Solución del Ejercicio Guiado:

Gráfica de Caída Libre

```
1 # Paso 1: Definir parámetros
2 y0 = 100 # altura inicial en metros
3 g = 9.81 # aceleración de gravedad en m/s²
4
5 # Calcular tiempo cuando toca el suelo: y = 0
6 # 0 = y0 - (1/2)*g*t² → t = sqrt(2*y0/g)
7 t_final = np.sqrt(2 * y0 / g)
8 print(f'Tiempo de caída: {t_final:.2f} segundos")
9 # Paso 2: Crear array de tiempo
10 t = np.linspace(0, t_final, 100)
11 # Paso 3: Calcular posiciones
12 y = y0 - 0.5 * g * t**2
13 # Paso 4: Crear la gráfica
14 plt.figure(figsize=(8, 6))
15 plt.plot(t, y, 'b-', linewidth=2, label='Posición vs Tiempo')
16 # Paso 5: Marcar punto de impacto
17 plt.plot(t_final, 0, 'ro', markersize=8, label='Impacto con el suelo')
18 # Paso 6: Personalizar la gráfica
19 plt.xlabel('Tiempo (s)')
20 plt.ylabel('Altura (m)')
21 plt.title('Caída Libre desde 100 metros')
22 plt.grid(True, alpha=0.3)
23 plt.legend()
24 # Agregar anotación
25 plt.annotate(f't = {t_final:.2f} s',
26             xy=(t_final, 0), xytext=(t_final-1, 20),
27             arrowprops=dict(arrowstyle='->', color='red'))
28 plt.show()
```

Matplotlib Avanzado

Matplotlib Avanzado $\ni$ Creando Varias Gráficas (Subplots)

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 x = np.linspace(0, 2*np.pi, 100)
5 y_sin = np.sin(x)
6 y_cos = np.cos(x)
7
8 fig, axs = plt.subplots(2, 1, figsize=(6,8))
9 axs[0].plot(x, y_sin, 'r-', label="sin(x)")
10 axs[0].legend()
11 axs[0].set_title("Seno")
12
13 axs[1].plot(x, y_cos, 'b--', label="cos(x)")
14 axs[1].legend()
15 axs[1].set_title("Coseno")
16
17 plt.tight_layout()
18 plt.show()
```

- `plt.subplots(rows, cols)`, retorna `fig` y un array de ejes `axs`.
- `plt.tight_layout()` ayuda a optimizar la separación entre subplots.

```
1 import matplotlib.pyplot as plt
2
3 etiquetas = ['A', 'B', 'C', 'D']
4 valores = [10, 25, 7, 15]
5
6 plt.bar(etiquetas, valores, color='lightblue')
7 plt.title("Gráfico de Barras")
8 plt.xlabel("Categorías")
9 plt.ylabel("Valores")
10 plt.show()
```

- Útil para datos categóricos o comparaciones discretas.
- `plt.barh` para barras horizontales.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 data = np.random.randn(1000) # 1000 valores normal estándar
5 plt.hist(data, bins=20, color='green', alpha=0.7)
6 plt.title("Histograma de datos aleatorios")
7 plt.xlabel("Valor")
8 plt.ylabel("Frecuencia")
9 plt.show()
```

- **bins** controla el número de barras en el histograma.
- **alpha**: transparencia de las barras.

Gráficos 3D Básicos

```
1 from mpl_toolkits.mplot3d import Axes3D # a partir de mpl_toolkits
2 import matplotlib.pyplot as plt
3 import numpy as np
4
5 fig = plt.figure()
6 ax = fig.add_subplot(111, projection='3d')
7 # ax ahora es un eje 3D
```

- Se puede usar `plt.subplots(subplot_kw={'projection':'3d'})` también.
- Métodos como `ax.plot3D`, `ax.scatter3D` están disponibles.

Gráficos 3D Básicos $\ni$ Plot de Superficie 3D (Ejemplo)

```
1 from mpl_toolkits.mplot3d import Axes3D
2 import matplotlib.pyplot as plt
3 import numpy as np
4
5 x = np.linspace(-5, 5, 50)
6 y = np.linspace(-5, 5, 50)
7 X, Y = np.meshgrid(x, y)
8 Z = np.sin(np.sqrt(X**2 + Y**2))
9
10 fig = plt.figure()
11 ax = fig.add_subplot(111, projection='3d')
12 ax.plot_surface(X, Y, Z, cmap='viridis')
13 ax.set_title("Superficie 3D")
14 plt.show()
```

- plot_surface, plot_wireframe, plot_contour, etc.

Introducción a pandas (Opcional)

Introducción a pandas (Opcional) $\ni$ ¿Por qué pandas?

- Librería para **manejo y análisis** de datos tabulares, muy usada en ciencia de datos.
- Estructuras **DataFrame** y **Series**.
- Integración con **NumPy** y **Matplotlib**.
- Lectura y escritura de archivos CSV, Excel, SQL, etc.

Introducción a pandas (Opcional) $\ni$ Ejemplo Rápido con DataFrame

```
1 import pandas as pd
2
3 datos = {
4     'Masa': [1.0, 3.5, 2.2],
5     'Velocidad': [10.0, 7.5, 12.3]
6 }
7 df = pd.DataFrame(datos)
8 print(df)
9
10 # Calcular Energía Cinética
11 df['E_c'] = 0.5 * df['Masa'] * (df['Velocidad']**2)
12 print(df)
```

- Facilita manipulación de datos y operaciones columnares.
- Opcional: graficar `df['E_c']` con `df.plot()` o `plt`.

Introducción a pandas (Opcional) $\ni$ Creando un Archivo CSV

```
1 import numpy as np
2 import pandas as pd
3
4 # Generar datos de trayectoria parabólica
5 tiempo = np.linspace(0, 5, 50)
6 pos_x = 15 * tiempo
7 pos_y = 20 * tiempo - 4.9 * tiempo**2
8
9 # Crear DataFrame y guardar
10 datos = {'Tiempo': tiempo, 'PosX': pos_x, 'PosY': pos_y}
11 df = pd.DataFrame(datos)
12 df.to_csv('trayectoria.csv', index=False)
13
14 print("Archivo 'trayectoria.csv' creado!")
15 print(df.head())
```

- **index=False**: no guarda índices como columna extra
- Archivo se crea en el directorio actual

Ejercicios Prácticos

Ejercicios Prácticos $\ni$ Ejercicio 1:

Subplots Seno, Coseno, Tangente



Enunciado

- Generar x en $[0..2\pi]$, con `np.linspace`.
- Crear 3 **subplots** en una columna:
 1. `sin(x)`
 2. `cos(x)`
 3. `tan(x)`
- Ajustar **ejes** y uso de **legends/titles**.
- Manejar la tangente en zonas cercanas a $\pi/2$ (**ver si se disparan valores**).
- Porque para `tan(x)` existe una linea vertical asintótica?

Ejercicios Prácticos $\ni$ Ejercicio 2: Comparando Histogramas y Barras



Enunciado

- Generar 100 valores aleatorios con `np.random.normal(5,1,100)` (media=5, sigma=1).
- Hacer un **histograma** de esos datos (`plt.hist`).
- Contar manualmente las frecuencias en cada bin, y crear un **gráfico de barras** para comparar.
- Observar similitudes y diferencias.
- Existen otras distribuciones distintas a parte de la **Normal**?

Objetivo: Entender la relación entre un histograma y los datos de frecuencia.



Enunciado

- Definir una **función** $f(x, y) = e^{-x^2-y^2}$ (campana).
- Usar `np.meshgrid` para crear `X, Y` en `[-2,2]`.
- Calcular $Z = f(X, Y)$.
- Crear un **plot de superficie** (`ax.plot_surface`) con un color map bonito.

Sugerencia: Ajustar `figsize` y `ax.set_zlim` para ver mejor la forma.

Ejercicios Prácticos $\ni$ Ejercicio 4: Leyendo Datos con pandas



Enunciado

- Tener un archivo CSV simple con columnas: **Tiempo**, **PosX**, **PosY** (p.e., trayectoria de un objeto).
- Usar `pandas.read_csv('archivo.csv')` para cargarlo a un DataFrame.
- Graficar **PosY** vs. **Tiempo** con **Matplotlib**.
- Agregar **labels** y título.

Objetivo: Introducir la idea de leer datos externos y visualizar.

Soluciones de Referencia

Soluciones de Referencia $\ni$ Solución 1 de Referencia:

Subplots Seno, Coseno, Tangente



```
1 # Generar array x en  $[0, 2\pi]$ 
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 x = np.linspace(0, 2*np.pi, 100)
6
7 # Calcular funciones trigonométricas
8 y_sin , y_cos , y_tan = np.sin(x), np.cos(x),
  ↳ np.tan(x)
9
10 # Limitar valores extremos de tangente para mejor
  ↳ visualización
11 y_tan = np.where(np.abs(y_tan) > 10, np.nan, y_tan)
12
13 # Crear subplots en una columna
14 fig, axs = plt.subplots(3, 1, figsize=(8, 10))
15
16 # Subplot 1: Seno
17 axs[0].plot(x, y_sin, 'r-', linewidth=2,
  ↳ label='sin(x)')
18 axs[0].set_title('Función Seno')
19 axs[0].set_ylabel('sin(x)')
20 axs[0].legend()
21 axs[0].grid(True, alpha=0.3)
```

```
22 # Subplot 2: Coseno
23 axs[1].plot(x, y_cos, 'b--', linewidth=2,
  ↳ label='cos(x)')
24 axs[1].set_title('Función Coseno')
25 axs[1].set_ylabel('cos(x)')
26 axs[1].legend()
27 axs[1].grid(True, alpha=0.3)
28
29 # Subplot 3: Tangente
30 axs[2].plot(x, y_tan, 'g:', linewidth=2,
  ↳ label='tan(x)')
31 axs[2].set_title('Función Tangente')
32 axs[2].set_xlabel('x (radianes)')
33 axs[2].set_ylabel('tan(x)')
34 axs[2].set_ylim(-5, 5) # Limitar rango y para
  ↳ mejor visualización
35 axs[2].legend()
36 axs[2].grid(True, alpha=0.3)
37
38 plt.tight_layout()
39 plt.show()
```

Discusión: El manejo de la tangente requiere cuidado especial en sus discontinuidades usando `np.where` y `set_ylim`.

Soluciones de Referencia $\ni$ Solución 2 de Referencia:

Comparando Histogramas y Barras



```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Generar datos aleatorios
5 datos = np.random.normal(5, 1, 100) # media=5,
↳ desv_std=1, 100 valores
6
7 # Crear figura con subplots
8 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12,
↳ 5))
9
10 # Histograma
11 n, bins, patches = ax1.hist(datos, bins=10,
↳ color='skyblue', alpha=0.7, edgecolor='black')
12 ax1.set_title('Histograma de Datos Aleatorios')
13 ax1.set_xlabel('Valor')
14 ax1.set_ylabel('Frecuencia')
15 ax1.grid(True, alpha=0.3)
16
17 # Calcular centros de bins para gráfico de barras
18 centros_bins = (bins[:-1] + bins[1:]) / 2
```

```
19 # Gráfico de barras usando las frecuencias del
↳ histograma
20 ax2.bar(centros_bins, n,
↳ width=(bins[1]-bins[0])*0.8,
↳ color='lightcoral', alpha=0.7,
↳ edgecolor='black')
21 ax2.set_title('Gráfico de Barras Equivalente')
22 ax2.set_xlabel('Valor (centro del bin)')
23 ax2.set_ylabel('Frecuencia')
24 ax2.grid(True, alpha=0.3)
25
26 plt.tight_layout()
27 plt.show()
28
29 print(f"Media de los datos: {np.mean(datos):.2f}")
30 print(f"Desviación estándar: {np.std(datos):.2f}")
31 print(f"Número total de valores: {len(datos)}")
```

Discusión: Los valores de frecuencia n del histograma se usan directamente para crear el gráfico de barras equivalente.

Soluciones de Referencia $\ni$ Solución 3 de Referencia:

Gráfica 3D de una Función



```
1 from mpl_toolkits.mplot3d import Axes3D
2 import matplotlib.pyplot as plt
3 import numpy as np
4
5 # Definir dominio para x e y
6 x = np.linspace(-2, 2, 50)
7 y = np.linspace(-2, 2, 50)
8
9 # Crear meshgrid
10 X, Y = np.meshgrid(x, y)
11
12 # Calcular la función  $f(x,y) = e^{-(x^2-y^2)}$ 
13 Z = np.exp(-(X**2 + Y**2))
14
15 # Crear figura 3D
16 fig = plt.figure(figsize=(10, 8))
17 ax = fig.add_subplot(111, projection='3d')
18
19 # Plot de superficie
20 surface = ax.plot_surface(X, Y, Z, cmap='plasma',
    ↪ alpha=0.9)
21
22 # Configurar etiquetas y título
23 ax.set_xlabel('X')
24 ax.set_ylabel('Y')
25 ax.set_zlabel('f(x,y) = e-(x²-y²)')
26 ax.set_title('Superficie 3D: Función Gaussiana 2D')
27
28 # Agregar barra de colores
29 fig.colorbar(surface, shrink=0.5, aspect=20)
30
31 # Ajustar límites del eje z
32 ax.set_zlim(0, 1)
33
34 # Ajustar ángulo de visualización
35 ax.view_init(elev=30, azimuth=45)
36 plt.show()
```

Discusión: La función gaussiana 2D crea una "campana" perfecta. El colormap **plasma** resalta bien las variaciones de altura.

Soluciones de Referencia $\ni$ Solución 4 de Referencia:

Leyendo Datos con pandas



```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 import numpy as np
4
5 # Primero, crear un archivo CSV de ejemplo
6 ↪ (opcional)
7 # Simular trayectoria de proyectil
8 t = np.linspace(0, 5, 50)
9 pos_x = 10 * t # velocidad constante en x
10 pos_y = 20 * t - 4.9 * t**2 # movimiento
11 ↪ parabólico en y
12
13 # Crear DataFrame y guardar como CSV
14 data_dict = {'Tiempo': t, 'PosX': pos_x, 'PosY':
15 ↪ pos_y}
16 df_ejemplo = pd.DataFrame(data_dict)
17 df_ejemplo.to_csv('trayectoria.csv', index=False)
18
19 # Leer datos desde CSV
20 df = pd.read_csv('trayectoria.csv')
21
22 # Mostrar primeras filas
23 print("Primeras 5 filas del DataFrame:")
24 print(df.head())
```

```
22 # Crear gráfica
23 plt.figure(figsize=(10, 6))
24 plt.plot(df['Tiempo'], df['PosY'], 'bo-',
25 ↪ linewidth=2, markersize=4)
26 plt.title('Trayectoria Vertical vs Tiempo')
27 plt.xlabel('Tiempo (s)')
28 plt.ylabel('Posición Y (m)')
29 plt.grid(True, alpha=0.3)
30
31 # Agregar anotaciones
32 plt.annotate('Altura máxima',
33 ↪ xy=(df['Tiempo'][df['PosY'].idxmax()],
34 ↪ df['PosY'].max()), xytext=(3, 15),
35 ↪ arrowprops=dict(arrowstyle='->', color='red'))
36 plt.show()
37
38 # Calcular y mostrar altura máxima y tiempo
39 print(f"Altura máxima: {df['PosY'].max():.2f} m")
40 print(f"Tiempo al que ocurre:
41 ↪ {df['Tiempo'][df['PosY'].idxmax():.2f} s")
```

Discusión: pandas facilita la lectura de datos externos y su manipulación. La integración con matplotlib es directa.

- Para **subplots** múltiples, considerar `figsize` y `tight_layout`.
- En **3D**, probar otras funciones como $\cos(\sqrt{x^2 + y^2})$ o $\sin(x * y)$.
- En **pandas**, puedes usar `df.describe()` para ver estadísticas rápidas.
- Explorar **paletas de colores**: `cmap='plasma'`, `cmap='coolwarm'`, etc.

- ¿Alguna confusión con **subplots** o configuración de ejes?
- ¿Problemas para instalar o importar **pandas**?
- ¿Dudas sobre gráficos 3D y el uso de **meshgrid**?

Levanta la mano o consulta abiertamente para que todos aprendan.

Conclusiones y Próximos Pasos

- Comparte tus resultados en **subplots**, **histogramas** o **3D**.
- Si usaste **pandas**, ¿cómo fue la experiencia de cargar datos y manipularlos?
- Observa **beneficios** de la visualización en la interpretación de datos físicos/matemáticos.

- **Matplotlib avanzado:**
 - Subplots múltiples, histogramas, barras, y nociones 3D.
- **Breve introducción a pandas** para datos tabulares.
- Integramos **NumPy**, **Matplotlib** y (opcionalmente) **pandas** para un flujo de trabajo más completo.
- Seguiremos perfeccionando la **visualización** y manipulación de datos en las próximas sesiones.

- **Semana 7, Sesión 2:** Profundizaremos en **visualizaciones personalizadas** (títulos, ejes, anotaciones, múltiples figuras) y exploraremos más sobre **manejo de datos**.
- Revisaremos **buenas prácticas** para proyectos colaborativos y la combinación de Python con LaTeX (si alcanza el tiempo).

Sigan practicando con datos reales o simulados para afianzar los conocimientos.

- **Matplotlib Docs** (especialmente ejemplos de galería).
- **pandas Documentation** (guía de 10 minutos).
- **Seaborn** (librería de visualización avanzada sobre Matplotlib).
- **Python Standard Library** - repaso general.

Gracias y hasta la próxima sesión

- Recuerden subir sus notebooks y experimentos a Google Drive o repositorio compartido.
- Practiquen diferentes tipos de gráficos e integren datos con **pandas** si pueden.